Windows x64 Assembly

Name: Matyas Abraham
IDNO: CTC-207-26

What Is Reverse Engineering?

Reverse engineering is the process of taking something that's already finished and figuring out how it actually works under the hood. Once you understand that, you can start looking for weaknesses in the logic, or in how it was built, and use that to your advantage. The general process is: find something worth attacking, figure out how it works, then use that knowledge for whatever you're trying to do.

Getting good at this takes a while, and there's a reason for that.

Why did it take so long?

Computers were never really designed to be human-friendly - they're designed to be fast. Assembly is basically the middle ground between the two: it's low-level enough for the CPU to execute quickly, but still readable enough that a person can follow what's happening, instruction by instruction (these instructions are called opcodes).

Tools

Prerequisites (Knowledge and Notes)

- Computer Science Concepts
- C/C++ *Experience*
- Assembly

What is a Protocol?

A protocol is basically a set of agreed-upon rules for how something should behave correctly - it shows up everywhere, not just computers, but in medicine, government, science, you name it. In this context it just means the standard way two systems are expected to talk to each other.

Example: FTP, HTTP, SMTP

Number Systems

Every number system has a base (also called a radix) - that's just how many unique digits it uses, including zero, before it rolls over to the next place value. Computers end up using a few different number systems depending on the situation, and once you're used to converting between them it stops being a big deal.

Binary (Base-2)

Digits used: 0, 1

This is the actual language of the hardware. Transistors act like tiny switches, and a low voltage reads as 0 (off) while a high voltage reads as 1 (on). One binary digit is a bit, and a group of 8 bits makes a byte. Each position is worth double the position before it: 2^0=1, 2^1=2, 2^2=4, 2^3=8, 2^4=16, 2^5=32, and so on.

Octal (Base-8)

Digits used: 0, 1, 2, 3, 4, 5, 6, 7

Not something you run into every day, but it's a handy shorthand for binary since exactly 3 bits map to a single octal digit (2^3 = 8). One place it's still around is Unix/Linux file permissions - things like chmod 755.

Decimal (Base-10)

Digits used: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

The normal counting system we use as humans. The computer does all its actual work in binary, but it converts everything to decimal before showing it to us, because that's what we're used to reading.

Hexadecimal (Base-16)

Digits used: 0-9 and A-F (A=10, B=11, C=12, D=13, E=14, F=15)

This is the shorthand of choice once you're reading assembly or memory addresses, because exactly 4 bits (a nibble) map to a single hex digit (2^4 = 16). That means one full byte only needs 2 hex characters instead of 8 binary digits, which is a lot easier on the eyes. You'll run into hex constantly - memory addresses like 0x7FFF, IPv6 addresses, and even web colors like #FFFFFF.

Challenge Questions

Q: ***What is 0xA in decimal?***
A: 10

Q: ***What is decimal 25 written in hexadecimal (with the 0x prefix)?***
A: 0x19

Bits and Bytes

Data type sizes change depending on the architecture, but the basic units stay the same:

- Bit - one binary digit. Either 0 or 1.
- Nibble - 4 bits.

- Byte - 8 bits.
- Word - 2 bytes.
- Double Word (DWORD) - 4 bytes. Twice the size of a word.
- Quad Word (QWORD) - 8 bytes. Four times the size of a word.

Data Type Sizes

Char - 1 byte (8 bits).

Int - comes in 16-bit, 32-bit and 64-bit flavors, though when people just say "int" they usually mean 32-bit. For signed integers, one bit is reserved to mark whether the number is positive or negative.

- Signed Int
  - 16-bit: -32,768 to 32,767
  - 32-bit: -2,147,483,648 to 2,147,483,647
  - 64-bit: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- **Unsigned Int** - minimum is 0, and the maximum is roughly double the signed maximum of the same size, since there's no bit reserved for the sign. A 32-bit unsigned int ranges from 0 to 4,294,967,295 - double the signed max of 2,147,483,647, but starting from 0 instead of a negative number.

Bool - technically only needs 1 bit since it's just true/false, but it still takes up a full byte. That's because CPUs are generally built to work in fixed-size chunks (1, 2, 4, 8 bytes) rather than addressing individual bits, which is covered more under alignment.

Challenge Questions

Q: ***How many bytes make up a WORD?***
A: 2 bytes

Q: ***How many bits make up a WORD?***
A: 16 bits

Binary Operations

True / False

In computing, false is just the value 0, and true is anything other than 0.

NOT (shown as "!")

NOT simply flips the bit.

- NOT 1 = 0
- NOT 0 = 1

AND (shown as "&")

AND checks whether both bits are 1. If they are, the result is 1 - otherwise it's 0.

- 1 AND 1 = 1
- 1 AND 0 = 0
- 0 AND 0 = 0

XOR (shown as "^")

XOR is 1 only if exactly one of the two bits is 1 (not both). Another way to think about it: it's checking whether the two bits are different from each other.

- 1 XOR 1 = 0
- 1 XOR 0 = 1
- 0 XOR 0 = 0

Challenge Questions

Q: ***What's the result of 1011 AND 1100?***
A: 1000

Q: ***What's the result of 1011 NAND 1100? (leading zeroes included)***
A: 0111

Registers

Registers are small storage slots built directly into the CPU. They're much faster to read and write than RAM, since the CPU doesn't have to go out over the bus to reach them. Things can differ a bit between 32-bit and 64-bit assembly - this write-up focuses on 64-bit Windows, since that's what the room covers.

General Purpose Registers

x64 has 16 general purpose registers: RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, and R8 through R15. Each one is 64 bits wide, but the older ones can also be accessed as smaller chunks - for example:

64-bit	32-bit	16-bit	8-bit
RAX	EAX	AX	AH / AL
RBX	EBX	BX	BH / BL
RCX	ECX	CX	CH / CL
RDX	EDX	DX	DH / DL

R8-R15 are the newer registers added when x64 came along (32-bit x86 only had 8 general purpose registers). They can be accessed in 64-bit (R8), 32-bit (R8D), 16-bit (R8W) and 8-bit (R8B) forms.

Special Purpose Registers

- **RSP** - the Stack Pointer. Always points at the top of the current stack.
- **RBP** - the Base Pointer. Usually marks the base of the current stack frame, so local variables and arguments can be reached with a fixed offset from it.
- **RIP** - the Instruction Pointer. Always points at the next instruction to execute. You can't write to it directly - it's the CPU's own way of tracking where it is in the program.

Challenge Questions

Q: ***How many bytes is RAX?***
A: 8 bytes (it's the full 64-bit register)

Q: ***How many bytes is EAX?***
A: 4 bytes (the lower 32-bit half of RAX)

Instructions

The whole point of a compiler is to turn the high-level code we write (C, C++, and so on) into something the CPU can actually run - that's Assembly. Every instruction does one very specific, small thing: move a value, add two numbers, compare two things. On its own an instruction barely does anything, but a real program is just thousands of these chained together. Even without the original source code, the assembly can always be pulled straight out of the compiled executable.

Some of the instructions that came up the most in this room:

- `MOV dest, src` - copies a value from src into dest. Doesn't touch any flags.

- `LEA dest, src` - loads the address of src into dest, instead of the value stored there. Comes up a lot for pointer arithmetic.
- `PUSH / POP` - PUSH puts a value on top of the stack and moves RSP down; POP takes it back off and moves RSP up.
- `ADD / SUB` - basic addition and subtraction.
- `MUL / IMUL` - multiplication (unsigned / signed).
- `DIV / IDIV` - division (unsigned / signed).
- `CMP` - compares two values by subtracting them internally and setting flags based on the result, without actually storing that result anywhere.
- `TEST` - similar idea to CMP, but uses AND instead of subtraction. Often used to check if a value is zero or to test a specific bit.
- `JMP` - jumps to another part of the code unconditionally.
- `JE/JZ, JNE/JNZ, JG, JL...` - conditional jumps. They check the flags left behind by the last CMP/TEST and only jump if the condition holds.
- `CALL / RET` - CALL pushes the return address on the stack and jumps into a function; RET pops that address back off and jumps back to it.
- `NOP` - does nothing for a cycle. Sometimes used for padding or alignment.

Reading assembly is really just a matter of going slowly and keeping track of what's sitting in each register and stack slot at every step.

Challenge Questions

Q: ***Which instruction returns from a function?***
A: RET

Q: ***Which instruction calls/executes a function?***
A: CALL

Q: ***Which instruction can save a register's value so it can be restored later?***
A: PUSH

Flags

Flags mark the result of whatever operation or comparison was just executed.

Status Flags

- **Zero Flag (ZF)** - set if the result of an operation is zero. Not set otherwise.

- **Carry Flag (CF)** - set if the last unsigned arithmetic operation carried (addition) or borrowed (subtraction) a bit beyond the register. Also gets set when a result would have been negative if it weren't for the operation being unsigned.
- **Overflow Flag (OF)** - set if a signed arithmetic operation produces a result too big for the register to hold.
- **Sign Flag (SF)** - set if the result of an operation is negative.
- **Adjust / Auxiliary Flag (AF)** - the same idea as the carry flag, but for Binary Coded Decimal (BCD) operations.
- **Parity Flag (PF)** - set to 1 if the number of set bits in the last 8 bits is even (10110100 → PF=1; 10110101 → PF=0).
- **Trap Flag (TF)** - allows single-stepping through a program, one instruction at a time.

Challenge Questions

Q: ***If two equal values are compared, what does the Zero Flag get set to?***
A: 1

Windows x64 Calling Convention

Windows has its own convention for how arguments get passed into a function - it's usually called the Microsoft x64 calling convention (sometimes just "fastcall"). It's not the same as the convention Linux uses (System V AMD64), which matters if you ever end up reversing something cross-platform.

The general rules:

- The first four integer/pointer arguments go into RCX, RDX, R8, and R9, in that order.
- The first four floating-point arguments go into XMM0-XMM3 instead. The important part is that the slot is based on the argument's position, not its type - so if argument 2 happens to be a float, it still uses the "2nd slot" (XMM1), and RDX just gets skipped rather than reused for something else.
- Anything past the first four arguments gets pushed onto the stack, right to left.
- The caller has to reserve 32 bytes of "shadow space" (also called home space) on the stack before every call, even if the function takes fewer than 4 arguments - this gives the callee somewhere to spill the register arguments if it needs to.
- The stack has to be 16-byte aligned at the exact point the CALL instruction runs.
- The return value comes back in RAX (or XMM0 for floating-point results).

Registers are also split up based on who's responsible for keeping their value intact:

- **Volatile (caller-saved):** RAX, RCX, RDX, R8, R9, R10, R11 - a called function is free to overwrite these, so if the caller still needs the value afterward, it has to save it beforehand.
- **Non-volatile (callee-saved):** RBX, RBP, RDI, RSI, RSP, R12-R15 - if a function uses any of these, it's responsible for saving the original value and restoring it before it returns.

Challenge Questions

Q: ***In fastcall, which 64-bit register holds a function's return value?***
A: RAX

Q: ***In fastcall, which register is the first function parameter passed in?***
A: RCX

Memory Layout

Every running process gets its own virtual address space, split into different regions depending on what's stored there:

- `.text` - the actual executable code. Normally marked read + execute, not writable, so a running program can't (easily) modify its own instructions.
- `.data` - global/static variables that already have a value assigned at compile time.
- `.rdata` - read-only data, like string constants.
- `.bss` - global/static variables that don't have an initial value. The OS just zeroes this region out when the program loads.
- **Heap** - memory allocated at runtime (malloc, HeapAlloc, new, etc.). Grows upward, toward higher addresses.
- **Stack** - holds local variables, arguments, and return addresses for each function call. Grows downward, toward lower addresses. Every function call pushes a new "stack frame", and it gets popped off again once the function returns.

Windows also keeps a couple of extra structures around per process/thread that come up a lot in lower-level work: the PEB (Process Environment Block) holds process-wide info like the list of loaded modules, and the TEB (Thread Environment Block) holds thread-specific info. On x64, the TEB is reachable through the GS segment register. These two show up constantly in shellcoding and malware analysis, since they're one of the ways code can resolve API addresses without calling something like GetProcAddress directly.

It's also worth knowing about ASLR (Address Space Layout Randomization), which randomizes where the stack, heap, and loaded modules end up in memory each time the process runs - specifically to make memory corruption exploits harder to write reliably.

Challenge Questions

Q: ***What order is data taken off of / put onto the stack? (acronym)***
A: LIFO (Last In, First Out)

Final Thoughts

Going through this room made a lot of things click that I only half understood before, especially registers and the calling convention. Raw assembly looked pretty intimidating at first, but once you actually know what each register is generally used for, and how a function call moves data around under the hood, it stops looking like random noise and starts looking like a pattern you can follow.

This is really just the starting point though. Actual reverse engineering means reading a lot more assembly than this, usually with no comments or symbol names to lean on, so the plan now is to keep practicing on small compiled programs in a disassembler and get faster at recognizing these patterns on sight.

Screenshot below shows the room completed at 100%: